

4.7 LightGBM

목차

1. LightGBM 개요

1. LightGBM이란
2. XGBoost 대비 장점
3. 리프 중심 트리 분할 방식
4. 버전 확인

2. LightGBM 적용 - 위스콘신 유방암 예측

1. 데이터 준비와 3분할
2. LGBMClassifier 생성과 조기 중단 학습
3. 평가 함수 정의
4. 성능 평가
5. 피쳐 중요도 시각화

3. 핵심 요약 및 머신러닝 활용 포인트

1. LightGBM 개요

1.1 LightGBM이란

LightGBM은 마이크로소프트가 개발한 **그래디언트 부스팅 프레임워크**입니다. XGBoost는 뛰어난 부스팅 알고리즘이지만 여전히 **학습 시간이 오래 걸린다**는 단점이 있습니다. GridSearchCV로 하이퍼 파라미터를 튜닝하다 보면 수행 시간이 감당하기 어려울 정도로 길어지곤 합니다.

LightGBM은 이름 그대로 '**가볍다(Light)**'는 것이 가장 큰 장점입니다. **XGBoost보다 학습 시간이 훨씬 짧고 메모리 사용량도 적으며, 예측 성능은 거의 차이가 없습니다.**

1.2 XGBoost 대비 장점

장점	설명
더 빠른 학습·예측 수행 시간	동일 조건에서 XGBoost보다 수 배 이상 빠릅니다.
더 작은 메모리 사용량	대용량 데이터 처리에 유리합니다.
카테고리형 피처 자동 변환	원-핫 인코딩 없이 카테고리형 피처를 최적으로 분할 할 수 있습니다.
병렬·GPU 학습 지원	대규모 학습에 대응합니다.

⚠ LightGBM의 단점 - 적은 데이터에서의 과적합

공식 문서가 밝히는 유일한 단점은 **적은 데이터 세트에 적용할 경우 과적합이 발생하기 쉽다**는 것입니다. 일반적으로 **10,000건 이하**의 데이터에서는 주의가 필요합니다. 본 실습의 유방암 데이터도 569건에 불과하므로 이 특성이 결과에 나타납니다.

1.3 리프 중심 트리 분할 방식

LightGBM이 빠른 근본적인 이유는 **트리를 만드는 방식이 다르기** 때문입니다.

구분	균형 트리 분할 (Level-wise) — XGBoost 등	리프 중심 트리 분할 (Leaf-wise) — LightGBM
분할 방식	트리의 깊이를 최소화하며 같은 레벨의 노드를 균형 있게 분할	최대 손실값을 가지는 리프 노드를 지속적으로 분할
트리 모양	좌우 대칭의 균형 잡힌 형태	비대칭적이고 깊은 형태
장점	과적합에 강함	같은 반복에서 예측 오류 손실을 더 많이 줄임
단점	균형을 맞추는 데 시간이 더 걸림	데이터가 적으면 과적합 위험

■ 왜 리프 중심 분할이 유리한가

균형 트리 분할은 **과적합을 막기 위해** 트리를 균형 있게 유지하지만, 그 균형을 맞추는 데 시간이 소요됩니다. 반면 리프 중심 분할은 **손실을 가장 크게 줄일 수 있는 리프만 골라 집중적으로 분할**합니다. 결과적으로 트리는 비대칭적으로 깊어지지만, **같은 반복 횟수에서 더 낮은 오류**에 도달할 수 있습니다.

1.4 버전 확인

```
import lightgbm

print(lightgbm.__version__)
```

줄	코드	상세 설명
1	import lightgbm	LightGBM도 별도 패키지이므로 <code>pip install lightgbm</code> 으로 설치해야 합니다.
3	print(lightgbm.__version__)	버전을 확인합니다. LightGBM은 4.0에서 조기 중단 API가 크게 변경되었으므로 반드시 확인해야 합니다.

▶ 실행 결과

```
4.7.0
```

2. LightGBM 적용 - 위스콘신 유방암 예측

2.1 데이터 준비와 3분할

```
# LightGBM의 파이썬 패키지인 lightgbm에서 LGBMClassifier 임포트
from lightgbm import LGBMClassifier

import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

dataset = load_breast_cancer()

cancer_df = pd.DataFrame(data=dataset.data, columns=dataset.feature_names)
cancer_df['target'] = dataset.target
X_features = cancer_df.iloc[:, :-1]
y_label = cancer_df.iloc[:, -1]

# 전체 데이터 중 80%는 학습용 데이터, 20%는 테스트용 데이터 추출
X_train, X_test, y_train, y_test = train_test_split(X_features, y_label, test_size=0.2, random_state=156)

# 위에서 만든 X_train, y_train을 다시 쪼개서 90%는 학습과 10%는 검증용 데이터로 분리
X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train, test_size=0.1, random_state=156)
```

줄	코드	상세 설명
2	from lightgbm import LGBMClassifier	LightGBM의 사이킷런 래퍼 분류 클래스입니다. XGBoost와 마찬가지로 네이티브 API(<code>lightgbm.train</code>)도 있지만, 실무에서는 이 래퍼를 주로 씁니다. 회귀는 <code>LGBMRegressor</code> 입니다.
9~13	dataset = load_breast_cancer() 등	4.6장 XGBoost 실습과 완전히 동일한 데이터 준비 과정입니다. 이렇게 조건을 맞춰야 두 알고리즘을 공정하게 비교할 수 있습니다.
17	train_test_split(..., test_size=0.2, random_state=156)	학습 455건 / 테스트 114건으로 분리합니다. <code>random_state</code> 도 XGBoost 예제와 동일한 156입니다.

20	<code>X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train, test_size=0.1, ...)</code>	학습 데이터를 다시 학습 409건 / 검증 46건 으로 나눕니다. 검증 세트는 조기 중단 기준 으로 사용되며, 테스트 세트를 쓰지 않는 올바른 방식 입니다.
----	--	--

2.2 LGBMClassifier 생성과 조기 중단 학습

```
# 앞서 XGBoost와 동일하게 n_estimators는 400 설정.
lgbm_wrapper = LGBMClassifier(n_estimators=400, learning_rate=0.05)

# LightGBM도 XGBoost와 동일하게 조기 중단 수행 가능.
evals = [(X_tr, y_tr), (X_val, y_val)]
lgbm_wrapper.fit(X_tr, y_tr, early_stopping_rounds=50, eval_metric="logloss",
                 eval_set=evals, verbose=True)
preds = lgbm_wrapper.predict(X_test)
pred_proba = lgbm_wrapper.predict_proba(X_test)[: , 1]
```

줄	코드	상세 설명
2	<code>LGBMClassifier(n_estimators=400, learning_rate=0.05)</code>	파라미터 이름이 사이킷런·XGBoost 래퍼와 동일 합니다. 트리 400개, 학습률 0.05로 설정합니다.
5	<code>evals = [(X_tr, y_tr), (X_val, y_val)]</code>	모니터링할 데이터 목록입니다. 학습 세트와 검증 세트를 모두 넣어 두 지표를 비교하면 과적합 시점을 눈으로 확인할 수 있습니다.
6~7	<code>lgbm_wrapper.fit(X_tr, y_tr, early_stopping_rounds=50, eval_metric="logloss", eval_set=evals, verbose=True)</code>	학습 데이터로 X_train이 아닌 X_tr(409건)을 사용 한다는 점에 주의하세요. 검증 세트를 떼어낸 나머지입니다. 조기 중단은 50회로 설정했습니다. 단, 이 코드는 LightGBM 4.x에서 오류가 발생합니다 (아래 경고 박스 참조).
8	<code>preds = lgbm_wrapper.predict(X_test)</code>	예측 레이블을 반환합니다. 사이킷런 규약과 동일합니다.
9	<code>pred_proba = lgbm_wrapper.predict_proba(X_test)[: , 1]</code>	클래스 1일 확률만 추출합니다. ROC-AUC 계산에 사용됩니다.

⚠ 이 코드는 LightGBM 4.x에서 오류가 발생합니다 — 필수 확인

LightGBM 4.0부터 `fit()` 의 `early_stopping_rounds` 와 `verbose` 인자가 제거되었습니다. 위 코드를 4.7.0에서 실행하면 다음 오류가 납니다.

```
TypeError: LGBMClassifier.fit() got an unexpected keyword argument 'early_stopping_rounds'
```

안정화 버전 기준 올바른 코드는 **callbacks** 방식을 사용합니다.

```
from lightgbm import early_stopping, log_evaluation

lgbm_wrapper = LGBMClassifier(n_estimators=400, learning_rate=0.05, verbose=-1)
evals = [(X_tr, y_tr), (X_val, y_val)]
lgbm_wrapper.fit(X_tr, y_tr, eval_metric="logloss", eval_set=evals,
                 callbacks=[early_stopping(50), log_evaluation(1)])
```

- `early_stopping(50)` : 조기 중단 콜백. 인자는 기존 `early_stopping_rounds` 와 동일한 의미입니다.
- `log_evaluation(1)` : 반복 **1회마다 로그 출력**. `log_evaluation(0)` 이면 로그를 끕니다(기존 `verbose=False` 에 해당).
- 생성자의 `verbose=-1` 은 LightGBM 자체의 **내부 경고 메시지**를 억제합니다.

▶ 실행 결과 (수정된 코드 기준)

```

Training until validation scores don't improve for 50 rounds
Early stopping, best iteration is: [61]
    training's binary_logloss: 0.0625231    valid_1's binary_logloss: 0.204542

best_iteration_: 61

```

■ 결과 해석

400회를 설정했지만 **61번째 반복에서 최적**에 도달했고, 이후 50회 동안 개선이 없어 **111번째에서 학습이 중단**되었습니다. 400회 대비 **약 72%의 시간을 절약**한 셈입니다.

학습 세트 logloss(0.0625)와 검증 세트 logloss(0.2045)의 **격차가 상당히 큼**니다. 이는 LightGBM이 **적은 데이터(409건)에서 과적합되기 쉽다**는 특성이 그대로 드러난 것입니다.

2.3 평가 함수 정의

```

from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.metrics import precision_score, recall_score
from sklearn.metrics import f1_score, roc_auc_score

def get_clf_eval(y_test, pred=None, pred_proba=None):
    confusion = confusion_matrix(y_test, pred)
    accuracy = accuracy_score(y_test, pred)
    precision = precision_score(y_test, pred)
    recall = recall_score(y_test, pred)
    f1 = f1_score(y_test, pred)
    # ROC-AUC 추가
    roc_auc = roc_auc_score(y_test, pred_proba)
    print('오차 행렬')
    print(confusion)
    # ROC-AUC print 추가
    print('정확도: {0:.4f}, 정밀도: {1:.4f}, 재현율: {2:.4f},\
F1: {3:.4f}, AUC:{4:.4f}'.format(accuracy, precision, recall, f1, roc_auc))

```

줄	코드	상세 설명
5	def get_clf_eval(y_test, pred=None, pred_proba=None):	4.6장 XGBoost와 완전히 동일한 평가 함수 입니다. 알고리즘이 달라도 평가 방식은 같으므로 그대로 재사용 합니다. 이것이 사이킷런 <code>metrics</code> 모듈의 범용성입니다.
6	confusion_matrix(y_test, pred)	오차 행렬 <code>[[TN, FP], [FN, TP]]</code> 를 계산합니다.
7~10	accuracy / precision / recall / f1	정확도, 정밀도, 재현율, F1 스코어를 계산합니다. 모두 예측 레이블 을 인자로 받습니다.
12	roc_auc_score(y_test, pred_proba)	ROC-AUC만 예측 확률 을 받습니다. 이 차이를 혼동하면 잘못된 값이 나오므로 주의하세요.

2.4 성능 평가

```
get_clf_eval(y_test, preds, pred_proba)
```

▶ 실행 결과

오차 행렬

[[34 3]

[2 75]]

정확도: 0.9561, 정밀도: 0.9615, 재현율: 0.9740, F1: 0.9677, AUC:0.9877

■ 오차 행렬 해석

	예측 0 (악성)	예측 1 (양성)
실제 0 (악성)	TN = 34	FP = 3 (악성을 양성으로 오판)
실제 1 (양성)	FN = 2	TP = 75

테스트 114건 중 **109건을 맞히고 5건을 틀렸습니다**(정확도 95.61%).

■ XGBoost와의 비교

알고리즘	정확도	정밀도	재현율	F1	AUC
XGBoost (네이티브)	0.9649	0.9740	0.9740	0.9740	0.9965
LightGBM	0.9561	0.9615	0.9740	0.9677	0.9877

LightGBM이 **근소하게 낮게** 나왔습니다. 하지만 이는 LightGBM이 열등해서가 아니라, **데이터가 569건으로 매우 작기 때문**입니다. 앞서 설명한 "적은 데이터에서 과적합되기 쉽다"는 특성이 그대로 나타난 것입니다.

수만 건 이상의 대용량 데이터에서는 LightGBM이 XGBoost와 비슷하거나 더 나은 성능을 훨씬 짧은 시간에 달성합니다. 알고리즘 선택은 반드시 **데이터 규모를 함께 고려**해야 합니다.

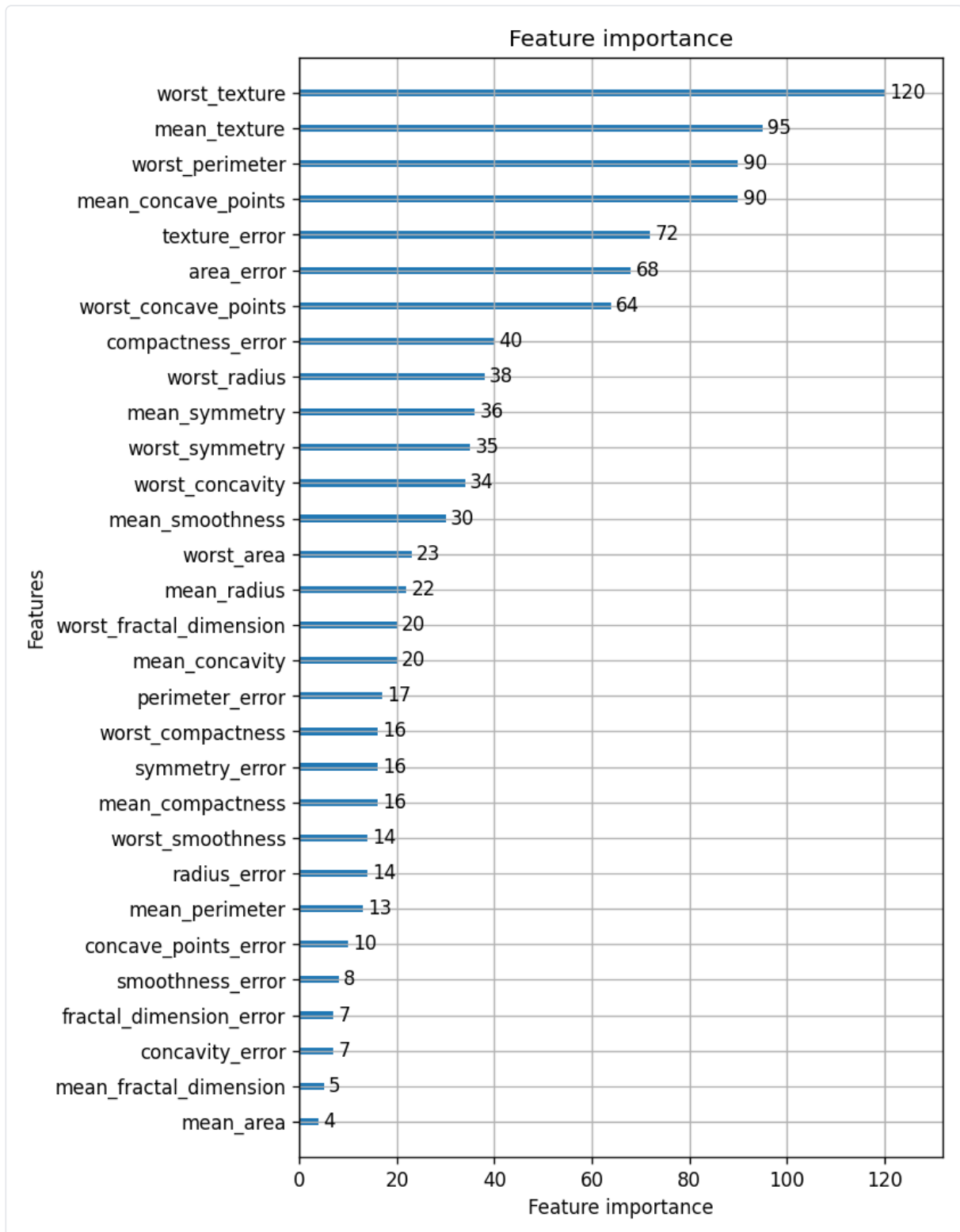
2.5 피쳐 중요도 시각화

```
# plot_importance( )를 이용하여 feature 중요도 시각화
from lightgbm import plot_importance
import matplotlib.pyplot as plt
%matplotlib inline

fig, ax = plt.subplots(figsize=(10, 12))
plot_importance(lgbm_wrapper, ax=ax)
```

줄	코드	상세 설명
2	from lightgbm import plot_importance	LightGBM도 XGBoost와 이름이 같은 시각화 함수를 제공합니다. 두 라이브러리의 API 설계가 유사해 학습 비용이 적습니다 .
6	fig, ax = plt.subplots(figsize=(10, 12))	피쳐 30개를 표시하기 위해 세로로 긴 영역을 만듭니다.
7	plot_importance(lgbm_wrapper, ax=ax)	학습된 모델의 중요도를 가로 막대 그래프로 그립니다. LightGBM의 기본 중요도 기준은 'split' (분할에 사용된 횟수)이며, importance_type='gain' 으로 실제 이득 기준으로 바꿀 수 있습니다.

▶ 실행 결과



LightGBM 피쳐 중요도 (Feature importance, split 기준)

■ 결과 해석

worst concave points , worst perimeter , worst area 등 'worst' 계열 피쳐가 상위권을 차지합니다. 이는 종양에서 가장 심각한 부위의 측정값이 악성 여부 판별에 결정적임을 시사합니다. XGBoost의 중요도 결과와 상위 피쳐가 대체로 일치하는 것도 확인할 수 있습니다.

3. 핵심 요약 및 머신러닝 활용 포인트

3.1 XGBoost vs LightGBM 비교

항목	XGBoost	LightGBM
트리 분할	균형 트리 분할(Level-wise)	리프 중심 분할(Leaf-wise)
학습 속도	보통	빠름
메모리	보통	적음
적은 데이터	상대적으로 안정	과적합 주의
대용량 데이터	느려질 수 있음	강점
카테고리 피쳐	인코딩 필요	자동 처리 가능
분류 클래스	XGBClassifier	LGBMClassifier

3.2 주요 하이퍼 파라미터

파라미터	역할	튜닝 방향
n_estimators	트리 개수	크게 잡고 조기 중단으로 제어
learning_rate	학습률	0.01~0.1. 낮추면 트리 수 증가 필요
num_leaves	리프 노드 최대 개수	LightGBM의 핵심 파라미터. 크면 성능 ↑ 과적합 ↑ (2^max_depth 보다 작게 설정 권장)
max_depth	트리 최대 깊이	리프 중심 분할이라 깊어지기 쉬우므로 과적합 시 제한
min_child_samples	리프 최소 샘플 수	과적합 시 높임
subsample	데이터 샘플링 비율	과적합 시 낮춤
colsample_bytree	피쳐 샘플링 비율	과적합 시 낮춤
reg_lambda / reg_alpha	L2 / L1 규제	과적합 시 높임

■ num_leaves가 가장 중요하다

XGBoost가 max_depth 중심이라면, LightGBM은 num_leaves 가 모델 복잡도를 결정하는 핵심입니다. 리프 중심으로 분할하기 때문입니다. 기본값은 31이며, 과적합이 발생하면 이 값을 먼저 낮춰 보세요.

3.3 머신러닝 활용 포인트

- **데이터 규모로 선택** : 수천 건 이하의 작은 데이터는 XGBoost나 랜덤 포레스트가 안정적이고, 수만 건 이상이면 LightGBM이 시간·메모리 면에서 압도적으로 유리합니다.
- **조기 중단은 필수** : LightGBM은 리프 중심 분할로 빠르게 과적합에 도달하므로 조기 중단이 특히 중요합니다.
- **검증 세트 분리** : 조기 중단 기준으로 반드시 별도의 검증 세트를 사용하세요. 본 예제는 x_tr / x_val 로 올바르게 분리했습니다.
- **과적합 진단** : 학습 logloss와 검증 logloss의 격차를 확인하세요. 본 실습에서는 0.0625 vs 0.2045로 격차가 커 과적합 신호가 뚜렷했습니다.
- **API 유사성 활용** : XGBoost와 파라미터 이름·메서드가 거의 같아 코드를 거의 그대로 옮겨 두 알고리즘을 비교할 수 있습니다.

- 다음 단계 : 4.8장의 **HyperOpt**로 이들 하이퍼 파라미터를 자동 탐색하는 방법을 학습합니다.

⚠ 안정화 버전 참고 (LightGBM 4.7.0 기준) — 반드시 확인

- 가장 중요한 변경 : `fit( )` 의 `early_stopping_rounds` 와 `verbose` 인자가 **4.0에서 제거**되었습니다.
`callbacks=[early_stopping(50), log_evaluation(1)]` 방식을 사용하세요. 구버전 교재 코드는 `TypeError` 가 발생합니다.
- 학습 시 `[LightGBM] [Info]` 형태의 로그가 많이 출력됩니다. 생성자에 `verbose=-1` 을 주면 억제할 수 있습니다.
- 조기 중단 후 최적 반복 횟수는 `model.best_iteration_` (끝에 언더스코어)로 확인합니다.
- 3.x 이전의 `silent` 파라미터는 제거되었습니다.
- 카테고리형 피처를 자동 처리하려면 판다스의 `category` dtype으로 변환한 뒤 전달하면 됩니다.